

AULA 07 • MÓDULO ESPECÍFICO PROFISSIONAL

Funções e Procedimentos

Reutilize código, divida problemas grandes em pequenas peças — o princípio que sustenta todo desenvolvimento profissional.



DURAÇÃO
4 horas



PROFESSOR
ALEXANDRE SANTANA



DATA
18/07/2026

RECAPITULANDO E AVANÇANDO

Onde paramos & para onde vamos



AULA ANTERIOR

O que vimos na Aula 06

- ✓ Vetores: declaração e acesso
- ✓ Percurso com for
- ✓ Estatísticas: maior, menor, média
- ✓ Matrizes bidimensionais
- ✓ Erros de índice e acumulador

HOJE VAMOS APRENDER

Objetivos da Aula

- Entender o que é uma função
- Distinguir função de procedimento
- Usar parâmetros e retorno de valor
- Compreender escopo de variáveis
- Introdução ao conceito de recursividade

ROTEIRO DE HOJE

Cronograma da Aula (4h)



1º BLOCO

50 min

Funções Conceito

Definição
e sintaxe



2º BLOCO

55 min

Parâmetros e Retorno

Entrada e
saída de funções



3º BLOCO

55 min

Escopo e Recursão

Local vs
global



4º BLOCO

60 min

Desafios Práticos

Funções
em JavaScript

O que é uma Função?

Definição

Uma função é um bloco de código com nome próprio que executa uma tarefa específica e pode ser chamado várias vezes. É como uma receita: define uma vez, usa quantas precisar.

Nome

Identificador único que a chama

Parâmetros

Dados de entrada que ela recebe

Retorno

Resultado que ela devolve

Exemplo: calcular área

```
funcao area(base, altura: real): real
```

```
inicio
```

```
    retorne base * altura
```

```
fimfuncao
```

```
// Chamada:
```

```
a <- area(5, 3) // retorna 15
```

```
escreva("Área:", a)
```

5 Conceitos Essenciais de Funções

Estes 5 conceitos formam o vocabulário completo para trabalhar com funções em qualquer linguagem:

funcao	proc.	param	return	escopo
função Retorna um valor ao ser chamada	proc. Executa ações, sem retornar valor	param. Variáveis de entrada da função	retorno Valor devolvido ao chamador	escopo Onde a variável existe e vale
<code>function</code>	<code>void</code>	<code>args</code>	<code>return</code>	<code>scope</code>

Função vs Procedimento

Toda função ou procedimento é um subprograma — a diferença está no retorno.

Função

Retorna valor	Sempre devolve um resultado
Expressão	Pode ser usada em expressões
Tipo	Precisa declarar tipo de retorno
Uso	resultado <- minhaFuncao()

Procedimento

Sem retorno	Apenas executa ações
Comandos	Ideal para imprimir, salvar, validar
void	Sem tipo de retorno declarado
Uso	chameMeuProcedimento()

Funções: Faça vs Evite

✓ FAÇA ✓

calcularMedia(notas)

Nome de verbo que descreve a ação

Uma função faz UMA coisa

Princípio da responsabilidade única

Parâmetros com nomes claros

base, altura — não b, h

Documente o que retorna

Comentário na declaração

Funções curtas (até 20 linhas)

Fáceis de ler e testar

! EVITE X

funcao1, f, calc

Nomes sem significado

Função que faz tudo

Divide em funções menores

Variável global dentro da função

Use parâmetros

Retornar sem valor em função

Declara tipo, retorna tipo

Recursão sem condição de parada

Loop infinito inevitável

Funções no VisuAlg (55 min)



Abra o VisuAlg — crie um arquivo `funcoes.alg` e implemente passo a passo

PASSO A PASSO

- 1 Declare a função ANTES do Algoritmo principal
- 2 Defina os parâmetros entre parênteses
`funcao soma(a, b: inteiro): inteiro`
- 3 Use retorne dentro do corpo da função
- 4 Chame passando os argumentos corretos
- 5 Teste com valores diferentes

O QUE OBSERVAR

Pontos importantes

- ✓ Função declarada fora do Início
- ✓ Variável local existe só dentro
- ✓ `retorne` encerra a função
- ✓ Pode chamar função dentro de função

Exercício guiado — Calculadora com Funções



Como construir

Crie uma função para cada operação:

- ✓ funcao somar(a, b: real): real
- ✓ funcao subtrair(a, b: real): real
- ✓ funcao multiplicar(a, b: real): real
- ✓ funcao dividir(a, b: real): real

⌚ Tempo: 25 minutos para concluir



Dicas importantes

Atenção a estes pontos:



Divisão por zero

Valide: se $b \neq 0$ antes de dividir



Tipo de retorno

Declare real pra resultado com decimais



Teste com 10 e 3

Soma=13, Sub=7, Mult=30, Div=3.33

Escopo de Variáveis



Escopo define onde uma variável existe e pode ser usada. Entender escopo evita um dos erros mais comuns dos iniciantes: a variável que "some" sem explicação. Todo conteúdo criado (textos, imagens, vídeos, código, design) tem um autor e é protegido por lei. Usar sem autorização é violação — mesmo na internet.

LOCAL	GLOBAL	PARÂMETRO
→ Existe só dentro da função → Declarada no Var da própria função → Destruída quando a função termina → Não afeta o código externo → Parâmetros são variáveis locais	→ Existe em todo o programa → Declarada no Var do algoritmo principal → Qualquer função pode ler e modificar → Use com cuidado — pode causar bugs	→ Cópia do argumento passado → Modificar o parâmetro não muda o original → Exceto passagem por referência → VisuAlg usa passagem por valor

Recursividade



Cuidado

Recursão

A função que chama a si mesma

O que é recursividade

Uma função recursiva é aquela que chama a si mesma para resolver uma versão menor do mesmo problema. Todo algoritmo recursivo precisa de uma condição de parada — sem ela, é um loop infinito.

Exemplo clássico — fatorial:

1. **Caso base** *se $n \leq 1$ então retorne 1*
2. **Chamada** *retorne $n * \text{fatorial}(n - 1)$*
3. **Trace** *$\text{fatorial}(3) \rightarrow 3 * \text{fatorial}(2) \rightarrow \dots$*
4. **Resultado** *$\text{fatorial}(5) = 120$*

4 Princípios para Boas Funções

Guia definitivo • Funções que seguem estes 4 princípios são *sempre* fáceis de testar, ler e reutilizar.

Os 4 princípios obrigatórios



Uma responsabilidade

Cada função faz exatamente uma coisa bem definida



Nome revela intenção

calcularMedia é melhor que func1 — qualquer dia



Entradas e saídas claras

Defina tipos dos parâmetros e do retorno



Curta e legível

Se precisa de scroll pra ler, quebre em funções menores

Erros Comuns com Funções

Os 3 erros mais comuns ao começar a trabalhar com funções:

 SEM RETORNO	 VARIÁVEL GLOBAL	 RECURSÃO INFINITA
Função com tipo de retorno declarado mas sem retorne — o compilador vai reclamar. SEMPRE RETORNE	Usar variável global dentro da função em vez de parâmetro — cria dependência oculta. USE PARÂMETROS	Esquecer a condição de parada — a função chama a si mesma para sempre. CASO BASE PRIMEIRO

Funções em JavaScript — A Transição



O que você aprendeu em pseudocódigo vai direto pro JavaScript. As palavras mudam, a lógica é idêntica.



01

Declaração em JavaScript

FAÇA

- ✓ `function nomeDaFuncao(param) { ... }`
- ✓ `const fn = (param) => { ... }`
- ✓ return em vez de retorne
- ✓ Sem tipo — JS é dinamicamente tipado

LEMBRE

Declare antes de chamar



02

Chamada e retorno

FAÇA

- ✓ `let resultado = minhaFuncao(10, 5)`
- ✓ `console.log(resultado)`
- ✓ Pode passar função como argumento

LEMBRE

Sem return = retorna undefined



03

Arrow function (atalho moderno)

FAÇA

- ✓ `const dobro = (n) => n * 2`
- ✓ `const soma = (a, b) => a + b`

LEMBRE

Arrow: return implícito sem chaves

Desafios com Funções (60 min)



Escolha 3 desafios

Implemente em VisuAlg ou JavaScript:

1

Conversor de temperaturas

Funções: *celsiusParaFahrenheit()* e *fahrenheitParaCelsius()*

2

Validador de dados

Funções: *ehPar()*, *ehPositivo()*, *ehMaiorDeIdade()*

3

Calculadora de IMC

Função *calcularIMC(peso, altura)* e *classificarIMC(imc)*



ENTREGA

45 min para fazer + 15 min apresentar

Como entregar:

**Salve o .alg ou .html no Google Drive da turma.
Demonstre chamando a função com 3 entradas
diferentes.**

O professor circula pela sala dando feedback enquanto você programa.

PRÓXIMA AULA • AULA 08

Registros, Teste de Mesa

- Estruturas de dados heterogêneas
- Campos com tipos diferentes
- Instâncias de registro
- Técnica do teste de mesa
- Revisão e projeto final

Excelente aula! Pratique as funções em casa. 🚀